

[S13]NPU上的PyTorch算子开发



LMJ1995

知识是有趣的

+ 关注他

27 人赞同了该文章

先理解我们要做的：写一个算子⁺，并使其被PyTorch调用

首先，“写算子”并不等于“写PyTorch算子”，一个加减乘除都可以视作一个算子，这样的算子和上层调用它的是 PyTorch/TensorFlow 还是其他框架都没有关系，这样的算子我们也可以将其称作一个Kernel。

一个这样的Kernel（核函数⁺、算子或者随便你怎么称呼或者理解这个概念，我们这里就将其称为Kernel或者算子，不做任何翻译）会操纵NPU/GPU上的内存搬运，调用NPU/GPU提供的最底层的接口，完成一个设计好的运算。

比如当前AddCustom，这里用到了文档中的Add接口。

这样的Kernel可以被直接调用，也可以通过PyTorch进行调用，接下来我们将打开这个过程，基于Gitee的算子[开发样例](Ascend/samples)，以一个PyTorch上的自定义加法算子⁺为例，对NPU上的算子调用流程进行全面的解剖。

基础的NPU上的PyTorch调用

我们通过一个标准化的镜像创建我们的工作环境，通过一小段代码的调用，我们可以看到目前torch-npu可以正常的工作在我们的环境下

```
import torch
import torch_npu

x = torch.randn(2, 2).npu()
y = torch.randn(2, 2).npu()

z = x.mm(y)
print(z)
```

此时的输出为

```
Warning: Device do not support double dtype now, dtype cast repalce with float.
tensor+([[ 0.9885,  0.8861],
          [-0.1908, -0.1112]], device='npu:0')
```

此时我们通过PyTorch在NPU上进行了一个简单的乘法，我们找到当前 `torch_npu` 的本地安装路径（可以通过 `pip show torch_npu` 来查看），在这个路径下可以找到 `npu_native_functions.yaml` 文件，这里包含了torch_npu已经帮我们完成适配的各个算子介绍，我们刚刚调用的乘法算子就在其中。

现在，我们要做的是在已经支持的PyTorch算子之外，额外创建一个PyTorch算子，基于PyTorch中创建自定义的算子事实上可以分成两部分，一是创建Kernel的实现，二是通过PyTorch创建对该Kernel的调用。

我们参考样例中对PyTorch算子的实现（`samples/operator/ascendc/0_introduction/1_add_frameworklaunch/PytorchInvocation`），我们现在什么都不做，尝试去执行 `test_ops_custom.py`，会产生以下的报错。

`test_ops_custom.py` 内容如下

```
import torch
import torch_npu
from torch_npu.testing.testcase import TestCase, run_tests

torch.npu.config.allow_internal_format = False

class TestCustomAdd(TestCase):

    def test_add_custom(self):
        length = [8, 2048]
        x = torch.rand(length, device='cpu', dtype=torch.float16)
        y = torch.rand(length, device='cpu', dtype=torch.float16)
        print(x, '\n', y)

        torch.npu.synchronize()
        output = torch_npu.npu_add_custom(x.npu(), y.npu()).cpu()
        torch.npu.synchronize()

        print(output)
        self.assertRtolEqual(output, x + y)

if __name__ == "__main__":
    run_tests()
```

报错为


```
Traceback (most recent call last):
  File "/home/aicc/00564131/op_dev/samples/operator/ascendc/0_introduction/1_add_custom.py", line 10, in <module>
    output = torch_npu.npu_add_custom(x.npu(), y.npu()).cpu()
AttributeError: module 'torch_npu' has no attribute 'npu_add_custom'
```

显然，这是因为当前的torch-npu⁺上还没有进行这个自定义Add算子的实现。

第一步，实现AddCustom

好在样例里直接提供了一个AddCustom的实现（samples/operator/ascendc/0_introduction/1_add_frameworklaunch/AddCustom），我们并不过多的在这里介绍单个算子是如何开发的，依托 1_add_frameworklaunch 这个目录下已经实现AddCustom进行后续的工作，先将其按照ReadMe安装

```
bash install.sh -v Ascend910A -i /usr/local/Ascend/ascend-toolkit/latest
```

这里发生了一个报错，参考blog.csdn.net/qq_45257411处理

成功执行后会生成一个run包，执行这个run包会将实现好的AddCustom安装至算子目录下。在执行安装前，我们不妨看下当前的情况是什么样的。

算子目录（执行 echo \$ASCEND_OPP_PATH 可以看到当前算子目录）下有一个 vendors 的文件夹，这里通常存放着我们创建的自定义的算子，当然了，这里面现在应该什么也没有。

回到我们的工作目录下吧，执行这个run包，我们再看看会发生什么！

```
(D910_PyTorch2.1) root@bms-compatible-0009:/home/aicc/00564131/op_dev/samples/operator/ascendc/0_introduction/1_add_frameworklaunch/AddCustom:
$ ./run.sh
Verifying archive integrity... 100%   SHA256 checksums are OK. All good.
Uncompressing version:1.0 100%
[runtime] [2024-12-23 17:37:31] [INFO] copy uninstall sh success
[ops_custom]upgrade framework
[runtime] [2024-12-23 17:37:31] [INFO] create /usr/local/Ascend/ascend-toolkit/latest/ops_custom/framework
[runtime] [2024-12-23 17:37:31] copy new ops framework files .....
[ops_custom]upgrade op proto
[runtime] [2024-12-23 17:37:31] [INFO] create /usr/local/Ascend/ascend-toolkit/latest/ops_custom/op_proto
[runtime] [2024-12-23 17:37:31] copy new ops op_proto files .....
[ops_custom]upgrade version.info
[runtime] [2024-12-23 17:37:31] copy new version.info files .....
[ops_custom]upgrade op impl
[runtime] [2024-12-23 17:37:31] [INFO] create /usr/local/Ascend/ascend-toolkit/latest/ops_custom/op_impl
[runtime] [2024-12-23 17:37:31] copy new ops op_impl files .....
[ops_custom]upgrade op api
[runtime] [2024-12-23 17:37:31] [INFO] create /usr/local/Ascend/ascend-toolkit/latest/ops_custom/op_api
[runtime] [2024-12-23 17:37:31] copy new ops op_api files .....
```



```
[runtime] [2024-12-23 17:37:31] [INFO] no need to upgrade custom.proto files
SUCCESS
(D910_PyTorch2.1) root@bms-compatible-0009:/home/aicc/00564131/op_dev/samples/c
total 20
dr-xrwxrwx 1 root root 4096 Dec 23 17:37 ./
drwxr-xr-x 1 root root 4096 Jul 26 12:16 ../
-rw-r----- 1 root root 24 Dec 23 17:37 config.ini
drwxr-x--- 6 root root 4096 Dec 23 17:37 customize/
(D910_PyTorch2.1) root@bms-compatible-0009:/home/aicc/00564131/op_dev/samples/c
```

我们会发现我们刚刚开发好的算子已经正确的安装到了vendors+这个自定义算子的路径下，在这里你可以找到 include/aclnn_add_custom.h 这个头文件+，这里面定义了一个 aclnnAddCustom 的Kernel实现，好了，你就这样完成了开发的第一部分的工作。

第二步，适配到PyTorch

完成了这部分的开发当然还远远不够，我们还需要对PyTorch进行一定的修改（如果你现在去执行 samples/operator/ascendc/0_introduction/1_add_frameworklaunch/PytorchInvocation 路径下的 test_ops_custom.py ，你还是会得到一样的报错）。

我们执行 samples/operator/ascendc/0_introduction/1_add_frameworklaunch/PytorchInvocation 下的 run_op_plugin.sh 就可以完成PyTorch的适配，这个脚本具体做了哪些事情呢？

首先是下载PTA源码仓，并更新submodule,其中torchair、op_plugin是本示例中依赖的submodule

```
git clone https://gitee.com/ascend/pytorch.git
git checkout -b v2.1.0 origin/v2.1.0
git submodule update --init --recursive
```

其次我们需要向 op_plugin_functions.yaml 中写入算子原型，注册pytorch自定义算子

```
FUNCTION_REGISTE_FIELD="op_plugin_patch/op_plugin_functions.yaml"
FUNCTION_REGISTE_FILE="${PYTORCH_DIR}/third_party/op-plugin/op_plugin/config/v2
line=" - func: npu_add_custom(Tensor x, Tensor y) -> Tensor"
if ! grep -q "\ $line" $FUNCTION_REGISTE_FILE; then
    sed -i "/custom:/r $FUNCTION_REGISTE_FIELD" $FUNCTION_REGISTE_FILE
fi
```

最后编译PTA插件并安装，其中需要将编写好的单算子kernel文件 AddCustomKernelNpuApi.cpp ，拷贝至op-plugin目录下编译(注： AddCustomKernelNpuApi.cpp 调用的正是我们刚刚开发的 aclnnAddCustom)，卸载当前的 torch-npu后重新安装新编译的torch-npu。


```
cp -rf op_plugin_patch/*.cpp ${PYTORCH_DIR}/third_party/op-plugin/op_plugin/ops
export DISABLE_INSTALL_TORCHAIR=FALSE
cd ${PYTORCH_DIR}
(bash ci/build.sh --python=${PYTHON_VERSION} --disable_rpc ; pip uninstall torch)
```

一个至关重要的部分就是 `AddCustomKernelNpuApi.cpp` 这个文件，可以看到这个文件使用 PyTorch中Aten的接口，定义了一个`npu_add_custom`的算法，可以说这个文件构成了PyTorch的调用和NPU之间的桥梁。

关于Aten：PyTorch的算子实现都在ATen函数集合里，开发NPU+PyTorch算子前，了解PyTorch上的算子运作原理是非常必要的。可以参考：[github.com/pytorch/pyto...](https://github.com/pytorch/pytorch)

在这之后重新执行 `test_ops_custom.py` 即可正常输出的到结果，证明我们当前进行的适配工作正常。

```
(D910_PyTorch2.1) root@bms-compatible-0009:/home/aicc/00564131/op_dev/samples/c
Fail to import hypothesis in common_utils, tests are not derandomized
<frozen importlib._bootstrap>:671: ImportWarning: TBEMetaPathLoader.exec_module
<frozen importlib._bootstrap>:914: ImportWarning: TEMetaPathFinder.find_spec()
<frozen importlib._bootstrap>:671: ImportWarning: TBEMetaPathLoader.exec_module
tensor([[0.1152, 0.9385, 0.7095, ..., 0.7500, 0.3130, 0.0044],
        [0.2759, 0.1240, 0.3550, ..., 0.7183, 0.3540, 0.5127],
        [0.6475, 0.8037, 0.6343, ..., 0.0840, 0.3560, 0.8677],
        ...,
        [0.7900, 0.2070, 0.7319, ..., 0.2363, 0.2803, 0.2510],
        [0.2993, 0.3140, 0.4355, ..., 0.8130, 0.3618, 0.5693],
        [0.3540, 0.7471, 0.9448, ..., 0.8877, 0.8691, 0.0869]],
        dtype=torch.float16)
tensor([[0.6689, 0.2119, 0.3105, ..., 0.6313, 0.9546, 0.7935],
        [0.0757, 0.8447, 0.2329, ..., 0.7256, 0.9160, 0.3975],
        [0.1968, 0.6567, 0.5322, ..., 0.3071, 0.8501, 0.0947],
        ...,
        [0.6748, 0.4189, 0.7202, ..., 0.0103, 0.6133, 0.3706],
        [0.1079, 0.3457, 0.7505, ..., 0.5947, 0.4390, 0.4434],
        [0.4102, 0.1792, 0.9648, ..., 0.6333, 0.5381, 0.6646]],
        dtype=torch.float16)
tensor([[0.7842, 1.1504, 1.0195, ..., 1.3809, 1.2676, 0.7979],
        [0.3516, 0.9688, 0.5879, ..., 1.4434, 1.2695, 0.9102],
        [0.8442, 1.4609, 1.1660, ..., 0.3911, 1.2061, 0.9624],
        ...,
        [1.4648, 0.6260, 1.4521, ..., 0.2466, 0.8936, 0.6216],
        [0.4072, 0.6597, 1.1855, ..., 1.4082, 0.8008, 1.0127],
        [0.7642, 0.9263, 1.9102, ..., 1.5215, 1.4072, 0.7515]],
        dtype=torch.float16)
.
-----
Ran 1 test in 8.721s
```

OK

(D910_PyTorch2.1) root@bms-compatible-0009:/

一些踩坑...

- 问题1: libgomp问题, 设置环境变量⁺即可: export LD_PRELOAD=/usr/lib/aarch64-linux-gnu/libgomp.so.1
- 问题2:执行PytorchInvocation下的run_op_plugin.sh失败: 不通过脚本进行编译和安装, 而是手动在PyTorch目录下进行安装
- 昇腾⁺的生态也是好起来了, 大部分的问题现在可以轻松的看着在网上找到解答, 和前几年有了质的变化LOL